Подробная инструкция к простой версии работ № 6 и № 7

ГИБДД, вариант 13. MySQL 8.0 и MySQL Workbench. Здесь одна процедура для шестой работы и две функции для седьмой. Для каждого шага указаны действие, ожидаемый результат, место снимка и объяснение для защиты. Следуйте разделам по порядку.

1 Что скачать и что сдавать

Файл	Назначение
Отчет_ЛР6_простая_версия.docx	Отдельный отчёт о процедуре; заполнить шесть мест снимками.
Отчет_ЛР7_простая_версия.docx	Отдельный отчёт о функциях; заполнить семь мест снимками.
01_lab6_procedure.sql	Установка одной простой процедуры.
02_lab7_functions.sql	Установка двух простых функций.
03_lab6_tests.sql	Проверки процедуры по рисункам отчёта № 6.
04_lab7_tests.sql	Проверки функций по рисункам отчёта № 7.
lab6_results.txt / lab7_results.txt	Вывод реального предварительного тестирования для сравнения.

Методичка разделяет эти темы: работа № 6 — подраздел 7.5, с. 131–133; работа № 7 — подраздел 7.6, с. 133–134. Поэтому оформлены два отчёта. Шестая допускает 1 –3 процедуры: выбрана одна. Для седьмой реализованы две функции: одна с ветвлением, другая с циклом. В коде есть входные параметры, локальные переменные, пользовательские @-переменные, цикл и проверки ошибочных значений.

Титульные листы в обоих отчётах перенесены без изменений из вашей пятой работы. На них сохранены старые № 5 и тема. Основные части явно подписаны № 6 и № 7. Это сохраняет согласованный старый титульник; если преподаватель потребует актуальные номер и тему на титуле, замените только эти строки.

Скачайте архив простой версии и распакуйте. Сначала откройте отчёты, чтобы видеть номера рисунков. Сохраните рабочие копии DOCX. Готовые блок-схемы заменять не требуется. Не смешивайте запросы этой версии с более сложными lr6_car_card и lr6_accidents_by_month из прежнего комплекта.

2 Подключение к базе и исходные данные

Откройте MySQL Workbench. На главной странице дважды щёлкните по тому подключению, которое использовали в пятой работе. Введите свой пароль MySQL.

В Navigator → Schemas нажмите Refresh, найдите gibdd и дважды щёлкните по имени. Откройте новую SQL-вкладку (обычно Ctrl+T).

Скопируйте запросы ниже, выделите нужный оператор и нажмите молнию Execute. Для текущего оператора обычно используется Ctrl+Enter, для выделенного фрагмента — Ctrl+Shift+Enter. Проверьте подсказку кнопки в вашей версии.

```
USE gibdd;
SELECT DATABASE() AS current_db, VERSION() AS mysql_version;
SHOW TABLES;
SELECT 'citizens' AS table_name, COUNT(*) AS rows_count FROM citizens
UNION ALL SELECT 'cars', COUNT(*) FROM cars
UNION ALL SELECT 'accidents', COUNT(*) FROM accidents
UNION ALL SELECT 'accident_cars', COUNT(*) FROM accident_cars
UNION ALL SELECT 'sales', COUNT(*) FROM sales;
```

Ожидается база gibdd и MySQL 8.0.x. Таблицы: citizens, cars, accidents, accident_cars, sales. В контрольном наборе соответственно 10, 10, 5, 10 и 5 строк. Если числа отличаются, выясните, сохраняли ли вы изменения в предыдущих работах. Не пересоздавайте таблицы поверх своей базы. При отсутствии gibdd сначала восстановите свой дамп работы № 4.

3 Схема данных — рисунок 1 в обоих отчётах

Если сохранена модель .mwb, откройте File → Open Model и EER Diagram. Если модели нет, выполните Database → Reverse Engineer, выберите своё подключение, затем схему gibdd и пять таблиц. Пройдите Next / Execute / Finish; включите размещение таблиц на диаграмме.

Раскройте EER Diagram. Расположите citizens слева, sales ниже, cars ближе к центру, accidents справа сверху, accident_cars между cars и accidents. Раздвиньте блоки, чтобы линии не пересекались.

Расширьте блоки за края: полностью должны читаться accident_cars, названия полей и типы. Проверьте шесть связей по таблице 1 отчёта. В accident_cars оба поля входят в первичный ключ.

Снимите область диаграммы через Win+Shift+S или инструмент своей системы. Этот собственный снимок можно использовать как рисунок 1 обоих отчётов: база одна и та же.

4 Работа № 6: установка простой процедуры

Выберите File → Open SQL Script, откройте 01_lab6_procedure.sql. Уберите выделение и выполните файл целиком обычной кнопкой-молнией. Внутренние строки CREATE PROCEDURE отдельно не запускайте.

В Action Output внизу проверьте успешное выполнение CREATE PROCEDURE. При первом запуске уведомление DROP PROCEDURE IF EXISTS о несуществующем объекте допустимо.

Обновите Schemas → gibdd → Stored Procedures. Найдите lr6_simple_year. Старые процедуры могут остаться — они не мешают, ничего удалять вручную не нужно.

Что делает файл: USE выбирает схему; DROP заменяет только lr6_simple_year; DELIMITER $$ позволяет передать тело с внутренними «;» целиком; CREATE сохраняет процедуру на сервере; DELIMITER ; возвращает обычный разделитель. Таблицы и записи не меняются.

5 Работа № 6: что происходит в алгоритме

Строка / переменная	Простое объяснение
p_year	Год, который передал пользователь. IN означает входной параметр.
v_month = 1	С какого месяца начинаем: январь.
v_count = 0	Здесь временно хранится число ДТП одного месяца.
v_total = 0	Здесь накапливается сумма за год.
@lr6_total = NULL	Убираем прошлый результат, чтобы при ошибке не прочитать старое число.
IF год неверен	Выдаём строку сообщения; в цикл не заходим.
WHILE v_month <= 12	Повторяем подсчёт, пока не обработаны все 12 месяцев.
SELECT COUNT(*) INTO v_count	Считаем ДТП нужного года и текущего месяца; записываем в переменную.
v_total = v_total + v_count	Прибавляем месяц к годовому итогу.
v_month = v_month + 1	Переходим к следующему месяцу; без этого цикл не завершится.
SELECT год, итог	Возвращаем пользователю одну строку.

Пример работы цикла для 2025 года:

Обработанный месяц	ДТП месяца	Накопленная сумма
1–7	0 в каждом	0
8	1	1
9	2	3
10	2	5
11–12	0 в каждом	5

Почему цикл, если можно написать один COUNT за год? Методичка требует программную реализацию цикла. Здесь 12 простых повторений показывают его работу без временной таблицы и курсора. Это учебная реализация для маленькой базы; для большой базы способ подсчёта стоит сравнить по производительности.

6 Работа № 6: список процедур — рисунок 3

```
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
```

```
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'PROCEDURE'
ORDER BY ROUTINE_NAME;
```

Выполните запрос. Должна быть строка lr6_simple_year / PROCEDURE. Если есть старые процедуры, оставьте их в списке — методичка просит список всех процедур базы. Снимите запрос и Result Grid и вставьте в рисунок 3 отчёта № 6. Рисунок 2 — готовый алгоритм.

7 Работа № 6: правильные вызовы — рисунки 4–6

Рисунок 4: год с происшествиями

```
CALL gibdd.lr6_simple_year(2025);
SELECT @lr6_total AS total;
```

Выполните CALL отдельно: одна строка report_year = 2025, accident_count = 5. Затем в том же соединении выполните SELECT: total = 5. Снимите оба результата. Если Workbench показывает только последнюю таблицу, переключитесь на вкладку результата CALL либо сделайте два снимка и вставьте их в одну рамку рисунка 4.

Рисунок 5: год без происшествий

```
CALL gibdd.lr6_simple_year(2024);
SELECT @lr6_total AS total;
```

Ожидается год 2024, accident_count = 0 и total = 0. Это правильный параметр с нулевым результатом, а не ошибка. Снимки вставьте в рисунок 5.

Рисунок 6: допустимые границы

```
CALL gibdd.lr6_simple_year(1901);
CALL gibdd.lr6_simple_year(2155);
```

Выполняйте CALL по одному. Для контрольной базы оба дают 0 ДТП. Сделайте два снимка в рамку рисунка 6. Оба года разрешены; диапазон задан нашим правилом в IF, а не ограничением DATE.

8 Работа № 6: неправильные вызовы — рисунок 7

```
CALL gibdd.lr6_simple_year(NULL);
SELECT @lr6_total AS total;
CALL gibdd.lr6_simple_year(1900);
SELECT @lr6_total AS total;
CALL gibdd.lr6_simple_year(2156);
SELECT @lr6_total AS total;
```

Запускайте по одному оператору. После каждого CALL ожидается обычная строка message = ERROR: year must be 1901..2155. После соответствующего SELECT — total = NULL. Красной ошибки в Action Output быть не должно: мы сознательно

выбрали простую диагностику через SELECT, без SIGNAL. NULL означает, что допустимый результат не вычислен.

В рисунок 7 вставьте результаты всех трёх параметров; можно несколько снимков в одной рамке. Они проверяют отсутствие года, выход ниже диапазона и выход выше диапазона. Не выполняйте между ошибочным CALL и SELECT другую процедуру: иначе прочитаете её итог.

9 Работа № 7: установка функций

Откройте File → Open SQL Script → 02_lab7_functions.sql. Выполните файл целиком, без выделения отдельных строк тела.

В Action Output должны успешно пройти оба CREATE FUNCTION. Обновите Schemas → gibdd → Functions. Нужны lr7_simple_car_count и lr7_simple_year_count.

Если MySQL сообщает о недостатке прав при CREATE FUNCTION с включённым binary logging, сохраните точный текст. Попросите преподавателя или администратора настроить права для учебной учётной записи. Не меняйте общесерверные настройки самостоятельно и не объявляйте функции DETERMINISTIC ради обхода ошибки.

READS SQL DATA означает чтение таблиц. NOT DETERMINISTIC означает, что после изменения таблиц результат для того же ID или года может стать другим. Обе функции возвращают INT. RETURN завершает вызов с одним числом; SELECT внутри тела с INTO только присваивает локальную переменную, а не выдаёт таблицу клиенту.

10 Работа № 7: список функций — рисунок 4

```
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'FUNCTION'
ORDER BY ROUTINE_NAME;
```

Снимите запрос и список с обеими функциями. Вставьте в рисунок 4 отчёта № 7. Рисунки 2 и 3 уже содержат алгоритмы.

11 Функция автомобиля: правильные параметры — рисунок 5

```
SET @car_id = 1;
SELECT @car_id AS carId,
       gibdd.lr7_simple_car_count(@car_id) AS accident_count;
SET @car_id = 9;
SELECT @car_id AS carId,
       gibdd.lr7_simple_car_count(@car_id) AS accident_count;
```

Выполняйте SET, затем соответствующий SELECT в том же соединении. Для автомобиля 1 ожидается 2, для 9 — 0. @car_id — пользовательская переменная,

значение которой передаётся параметру p_car_id. Снимки обоих SELECT вставьте в рисунок 5.

Как объяснить: сначала проверяем ID, затем отдельно существование автомобиля. Только после этого считаем строки accident_cars. В существующем автомобиле без ДТП COUNT равен нулю, поэтому 0 — нормальный результат. Участие одного автомобиля в одном ДТП не повторяется благодаря составному первичному ключу.

12 Функция автомобиля: ошибки — рисунок 6

```
SELECT gibdd.lr7_simple_car_count(NULL) AS null_id,
       gibdd.lr7_simple_car_count(0) AS zero_id,
       gibdd.lr7_simple_car_count(-1) AS negative_id,
       gibdd.lr7_simple_car_count(999999) AS missing_id;
```

Ожидаются четыре столбца со значением -1. Один снимок SELECT и Result Grid вставьте в рисунок 6. NULL, 0 и отрицательный код отвергает первая проверка. Положительный, но отсутствующий ID 999999 отвергает проверка существования. Если в вашей базе он есть, выберите другой действительно отсутствующий положительный ID и поправьте тест и отчёт согласованно.

-1 — оговорённый признак ошибки. Это не число ДТП и не SQL-исключение. Функция может вернуть только число одного объявленного типа, поэтому мы не возвращаем вместо INT текстовое сообщение. При использовании такого результата в программе сначала нужно проверить, не равен ли он -1.

13 Функция года: правильные параметры — рисунок 7

```
SET @report_year = 2025;
SELECT @report_year AS report_year,
       gibdd.lr7_simple_year_count(@report_year) AS accident_count;
SET @report_year = 2024;
SELECT @report_year AS report_year,
       gibdd.lr7_simple_year_count(@report_year) AS accident_count;
```

Ожидаются 5 и 0. Сделайте два снимка в рисунок 7. Алгоритм цикла уже знаком из процедуры: 12 месяцев, COUNT, сложение, следующий месяц. Вместо SELECT с итогом функция выполняет RETURN v_total. Пользователь вызывает её внутри SELECT, а не через CALL.

14 Функция года: границы и ошибки — рисунки 8–9

Рисунок 8: обе допустимые границы

```
SELECT gibdd.lr7_simple_year_count(1901) AS min_year,
       gibdd.lr7_simple_year_count(2155) AS max_year;
```

Ожидаются 0 и 0. Сделайте один снимок в рисунок 8.

Рисунок 9: значения вне диапазона

```
SELECT gibdd.lr7_simple_year_count(NULL) AS null_year,
       gibdd.lr7_simple_year_count(1900) AS below_min,
       gibdd.lr7_simple_year_count(2156) AS above_max;
```

Ожидаются три значения -1. Сделайте один снимок в рисунок 9. Ошибочная ветвь возвращает -1 до входа в цикл; корректный, но пустой год возвращает 0 после подсчёта. Это различие обязательно объясните на защите.

15 Как заполнить отчёты

Откройте соответствующий DOCX. Подсказка над каждой рамкой говорит, какой результат нужен. Номера рисунков в двух отчётах независимы.

Внутри нужной рамки выделите текст «МЕСТО ДЛЯ ВАШЕГО СКРИНШОТА» и вставьте своё изображение через Вставка → Рисунки или из буфера. Обтекание выберите «В тексте».

Если в одну рамку требуется несколько результатов, вставьте картинки друг под другом. Минимальная высота ячейки увеличится. Сохраняйте пропорции и не уменьшайте текст до нечитаемого размера.

Подпись «Рисунок N — …» оставьте. Подсказку «Вставить свой снимок…» над заполненной рамкой удалите. Готовые блок-схемы и их подписи сохраните.

Сверьте числа с собственной базой. В шаблоне указаны ожидаемые значения контрольного набора. После проверки замените формулировки о необходимости вставки снимков на описание фактически выполненных тестов.

Сохраните DOCX, затем экспортируйте в PDF и просмотрите все страницы: сетки таблиц, подписи слева без отступа, читаемость кода и изображений. Старый титульник должен остаться на первой странице.

16 Короткие объяснения для защиты

Вопрос	**Ответ своими словами**
Что делает процедура?	Получает год, проверяет его, обходит 12 месяцев, суммирует ДТП и выводит строку.
Чем функция отличается?	Возвращает одно число через RETURN; её можно поставить внутрь SELECT. Процедуру запускаю CALL.
Где ветвление?	IF проверяет год или код. Ошибочная ветка не выполняет подсчёт.
Где цикл?	WHILE идёт от месяца 1 до 12. Каждый раз считаю месяц, прибавляю к сумме и увеличиваю номер месяца.
Где локальные переменные?	DECLARE v_month, v_count, v_total и v_exists. Они существуют внутри вызова.

Вопрос	Ответ своими словами
Где глобальные переменные?	В учебном смысле это @lr6_total, @car_id и @report_year вне локального блока. Технически они сессионные, доступны только в текущем соединении. @@GLOBAL сервера не меняю.
Почему ошибка функции -1, а не 0?	Ноль может означать отсутствие ДТП. Отрицательное количество невозможно, поэтому -1 однозначно обозначает неверный параметр.
Почему функция не DETERMINISTIC?	Она читает таблицы. Если добавить ДТП, тот же параметр даст другое число.
Зачем проверять существование автомобиля?	Чтобы отсутствующий ID не выглядел как существующая машина без ДТП.
Как понимаю итог 5?	В 2025 году есть 1 ДТП в августе, 2 в сентябре и 2 в октябре: всего 5. Считаю сами ДТП, не 10 машин-участников.

17 Если что-то не работает

Проблема	Что проверить
Unknown database / Table does not exist	То ли подключение открыто? Есть ли gibdd из работы № 4? Проверьте SHOW TABLES.
Procedure / Function does not exist	Выполнен ли целиком файл установки? Обновите схему и проверьте имя с префиксом simple.
Ошибка синтаксиса около DECLARE или END	Не выполняйте строки тела отдельно. Запускайте весь установочный файл вместе с DELIMITER.
@-переменная равна NULL после удачного вызова	CALL, SET и SELECT должны быть в одном соединении; проверьте, что не переподключались.
Функция вернула -1	Проверьте параметр: для отрицательных тестов это правильный результат.
Процедура показывает message с ERROR	В простом варианте это нормальная строка результата проверки. После ошибочного года @lr6_total = NULL.
Значения отличаются от 5, 2 или 0	Ваша база могла измениться. Сверьте даты ДТП и участия; не подгоняйте скриншоты под шаблон.
Недостаточно прав для CREATE FUNCTION	Сохраните сообщение, обратитесь к преподавателю / администратору. Характеристики функций не подменяйте.

18 Последняя проверка

- ☐ У процедуры показаны 2025, 2024, обе границы и три неверных года.
- ☐ У первой функции показаны две нормальные машины и четыре неверных ID.
- ☐ У второй функции показаны два обычных года, две границы и три неверных года.
- ☐ Списки процедур и функций сняты отдельно; схема есть в каждом отчёте.
- ☐ Вставлены все 6 рамок работы № 6 и все 7 рамок работы № 7.
- ☐ Подсказки удалены, собственные результаты совпадают с текстом.
- ☐ Старый титульник сохранён; отчёты и итоговые PDF просмотрены.